

DocumentSync

A General-Purpose Existing-Document Editing and Synchronisation Platform

COMS3011A Student Project Proposal

Dean Feldman (2803899)

Group members: *To be confirmed*

July 20, 2026

Abstract

Many organisations already possess collections of Word documents or PowerPoint presentations that contain repeated or closely related content. A paragraph, bullet point, table entry, heading, or slide element may appear in several files, while names, dates, addresses, identifiers, and other case-specific values differ. Updating the same content manually across all files is slow and can introduce inconsistencies, missed changes, and copy-and-paste errors.

This proposal presents DocumentSync, a general-purpose web application that allows users to upload their existing documents, identify matching or similar content, confirm which items should be linked, edit shared content once, preview the result, and produce updated copies of all selected files. The system will preserve each document's unique information and keep the original uploads unchanged.

The initial implementation will prioritise reliable DOCX support and user-confirmed matching. PowerPoint support, more advanced formatting preservation, automatic suggestions, approvals, and cloud-storage workflows will be added only after the core workflow is stable. The project will include a separate frontend and backend, a team-designed hand-written HTTP API, established authentication, a relational database, automated testing, CI/CD, a relevant external API integration, and a public documentation website.

Contents

1 Problem Statement	4
2 Proposed Solution	4
2.1 Example	5
3 General Applicability	5
4 Target Users and Stakeholders	5
5 Project Objectives	6
6 Scope and Feature Tiers	6
6.1 Basic Tier: Minimum Viable Product	6
6.2 Intermediate Tier: Functional Product	7
6.3 Advanced Tier: Market-Ready Product	7
7 How the Project Meets the Compulsory Requirements	8
8 Initial Technical Design	8
8.1 Proposed Technology Stack	8
8.2 Major Components and Responsibilities	9
8.3 High-Level Data Flow	9
9 Preliminary Database Design	10
10 Security and Privacy	10
11 Project Methodology	11
11.1 Definition of Done	11
12 Milestone-Aligned Development Plan	12
12.1 Milestone 1: Foundations	12
12.2 Milestone 2: Core Product	12
12.3 Milestone 3: Stabilisation	12
12.4 Final Submission	13
13 Initial User Stories	13
13.1 Upload Existing Documents	13
13.2 Confirm Shared Content	13
13.3 Edit Once Across Files	13
13.4 Download Updated Versions	14
14 Testing Strategy	14
15 Risks and Mitigations	14
16 Alternatives Considered	15
16.1 Manual Search and Replace	15
16.2 Rebuilding Documents as Templates	15
16.3 Fully Automatic Matching and Editing	16
16.4 Direct Database or Storage Access From the Frontend	16

17 Expected Deliverables	16
18 AI Use and Attribution	16
19 Conclusion	17
A Initial Acceptance Test Scenario	17
B Items Requiring Stakeholder Confirmation	17

1 Problem Statement

Organisations often maintain many documents that are mostly the same but are customised for different clients, cases, buildings, projects, departments, products, employees, courses, or locations. The documents already exist and may contain carefully prepared formatting, tables, lists, logos, headings, and domain-specific wording.

When shared content changes, users must manually open every affected file, locate the relevant content, apply the edit, and verify that document-specific details remain correct. This process is:

- repetitive and time-consuming;
- vulnerable to missed updates;
- vulnerable to incorrect search-and-replace operations;
- difficult to audit;
- difficult to verify across large document sets;
- likely to create inconsistent versions;
- unsuitable for repeated updates across many similar files.

For example, an organisation may have separate agreements for Building A, Building B, and Building C. Each document contains a similar monthly-reporting clause, but each file also contains unique building details. If the reporting clause changes, the user must update all three documents without accidentally changing the unique details.

The proposed system will allow the existing files to be uploaded and edited as a coordinated set rather than recreated from scratch.

2 Proposed Solution

DocumentSync will be a web-based document comparison, editing, and synchronisation platform. The system will support a controlled workflow in which users remain responsible for confirming what content is linked.

The main workflow will be:

1. The user uploads existing documents.
2. The backend extracts supported document structure and text.
3. The system displays paragraphs, headings, list items, table cells, or slide elements.
4. The system suggests exact or similar content that may be related.
5. The user confirms which content should be linked.
6. The user edits the linked content once.
7. The system applies the change to every selected document.
8. Document-specific values are preserved.
9. The user previews all affected files and changes.
10. The user confirms the operation.
11. The system creates new downloadable versions and preserves the originals.

The system will not silently assume that similar content is equivalent. Automatic matching will produce suggestions only. A user must confirm the relationship before a bulk edit can be applied.

2.1 Example

The user uploads:

- `Building-A-Agreement.docx`
- `Building-B-Agreement.docx`
- `Building-C-Agreement.docx`

Each document contains:

“The building manager must submit the report every month.”

The user changes the linked paragraph to:

“The building manager must submit the report by the fifth working day of every month.”

The system updates the confirmed paragraph in all three files while preserving each document’s building name, address, dates, contacts, and unrelated content.

3 General Applicability

The platform will not be tied to a specific topic or industry. It may be used for:

- client agreements;
- project reports;
- building or site documents;
- departmental policies;
- legal or administrative case files;
- educational documents;
- product documentation;
- employee documents;
- presentations for different audiences;
- repeated operational forms or reports.

The core concept is the same in every case: users already have multiple related files and need to apply the same controlled change across them.

4 Target Users and Stakeholders

Potential users include:

- **Document editor:** uploads files, confirms matches, makes changes, and downloads updated versions;
- **Reviewer:** checks previews and approves changes before they are applied;

- **Administrator:** manages users, roles, organisations, retention settings, and supported document policies;
- **Operational user:** performs routine updates across document sets;
- **Document owner:** owns or is responsible for the content being edited;
- **Academic client or tutor:** reviews progress and provides project feedback where applicable.

Stakeholder requirements, feedback, and test results will only be reported when they have actually been gathered.

5 Project Objectives

The project aims to:

1. reduce repeated manual editing across existing files;
2. identify exact and similar content across document sets;
3. require user confirmation before content is linked;
4. allow one edit to be applied across multiple confirmed locations;
5. preserve unique document-specific information;
6. preserve formatting as reliably as practical;
7. preview changes before producing updated files;
8. keep original uploads unchanged;
9. maintain version and audit information;
10. provide a secure, accessible, responsive, tested, deployed, and documented product.

6 Scope and Feature Tiers

6.1 Basic Tier: Minimum Viable Product

- Registration, login, logout, password reset, and account deletion using an established authentication provider or library;
- Upload two or more DOCX files;
- Validate supported file types and size limits;
- Extract and display paragraphs and basic text structure;
- Identify exact matching paragraphs;
- Allow users to manually select and link matching content;
- Edit linked content once;
- Preview changes in every affected document;
- Generate updated DOCX copies;
- Preserve the original uploaded files;
- Store document-set metadata, versions, and change history;
- Responsive and accessible interfaces;

- Separate frontend and backend applications;
- A hand-written HTTP API designed by the team;
- A relational database accessed through the backend;
- CI/CD for tests, builds, collaboration, and deployment;
- A relevant external API integration;
- A public documentation website.

6.2 Intermediate Tier: Functional Product

- Similarity-based suggestions for paragraphs that are not exactly identical;
- Support for headings, lists, and simple tables;
- Side-by-side before-and-after comparison;
- Highlighting of every affected location;
- Placeholders for document-specific values such as names, dates, and addresses;
- Reusable confirmed link groups;
- Undo and rollback to a previous generated version;
- Batch download of updated files;
- Search, filtering, and status tracking for document sets;
- Reviewer approval workflow;
- Automated backend, document-processing, and user-interface tests;
- Formal user testing with recorded and evaluated feedback;
- Failure handling for invalid, corrupt, password-protected, or unsupported files.

6.3 Advanced Tier: Market-Ready Product

- PowerPoint support for text boxes, titles, bullet lists, and selected table content;
- More reliable preservation of styles, numbering, tables, and layout;
- Impact analysis before a bulk edit is applied;
- Fine-grained permissions for organisations, document sets, editors, and reviewers;
- Complete audit records showing actor, action, timestamp, and affected files;
- Approval states such as draft, under review, approved, rejected, and superseded;
- Secure cloud-storage integration;
- Notifications for review requests and completed edit jobs;
- Performance optimisation for large files and document sets;
- Accessibility testing using automated and manual techniques;
- Production monitoring, structured error reporting, backups, and recovery documentation.

7 How the Project Meets the Compulsory Requirements

Requirement	Planned implementation
Git-compatible version control	All source code and documentation will be stored in Git repositories. Every member will make meaningful contributions through issues, branches, commits, pull requests, reviews, tests, and documentation.
Documented Git methodology	The team will document branch naming, issue linking, commit conventions, pull-request expectations, code-review rules, merge procedures, release tags, and hotfix handling.
Responsive and accessible design	The frontend will support mobile, tablet, and desktop layouts. It will use semantic HTML, accessible labels, keyboard navigation, visible focus states, sufficient contrast, and clear validation and error messages.
CI/CD	The pipeline will run linting, type checks, unit tests, document-processing tests, backend integration tests, frontend tests, builds, and deployment checks.
Separate frontend and backend	The frontend will manage interaction and previews. The backend will manage file processing, validation, authorisation, persistence, versioning, and document generation.
Hand-written HTTP API	The team will design and implement the backend HTTP interface used by the frontend. Managed databases or storage services will not replace the team's own application endpoints.
Established authentication	Authentication will use an established provider or maintained library. The team will not implement password hashing, password-reset tokens, or session security from scratch.
Account lifecycle	The application will support registration, login, logout, password reset, and account deletion. Data-retention and deletion behaviour will be documented.
External API integration	The preferred integration is a cloud-storage service such as Google Drive, OneDrive, or SharePoint for importing or exporting approved document versions.
Public documentation website	A public documentation site will cover architecture, setup, database design, backend interface, deployment, testing, methodology, security, accessibility, file processing, and AI use.
Required documentation	The project will document architecture, database design, deployment, setup, testing, dependencies, methodology, Git practices, security, accessibility, external integration, decisions, and AI use.

8 Initial Technical Design

8.1 Proposed Technology Stack

The final stack will be agreed by the group and justified through architecture decision records. A suitable low-risk option is:

- **Frontend:** React with TypeScript and Vite;
- **Backend:** Node.js with TypeScript and Express or Fastify;
- **Database:** PostgreSQL;

- **File storage:** private object storage or a secured server-side storage service;
- **Data access:** a maintained query builder or ORM used only for database access;
- **Authentication:** an established provider or maintained framework integration;
- **DOCX processing:** a maintained ZIP/XML or DOCX manipulation library;
- **PPTX processing:** a maintained presentation-processing library, if added after DOCX support;
- **Frontend testing:** Vitest and Playwright;
- **Backend testing:** Vitest or Jest with HTTP-level integration tests;
- **Documentation:** Docusaurus, MkDocs, or a similar Markdown-based tool;
- **CI/CD:** GitHub Actions or an equivalent Git-integrated pipeline.

The document-processing library must be selected only after an early prototype demonstrates acceptable formatting preservation on representative sample files.

8.2 Major Components and Responsibilities

1. **Web client:** uploads, document-set management, match review, editing, previews, approvals, accessibility, and responsive interaction;
2. **Backend application:** authentication integration, validation, authorisation, orchestration, persistence, versioning, audit events, and secure file access;
3. **Document parser:** extracts supported text and structural elements from uploaded files;
4. **Matching engine:** identifies exact matches and produces similarity suggestions;
5. **Edit engine:** applies confirmed changes to selected document elements;
6. **Preview and difference engine:** displays affected locations and before-and-after content;
7. **Relational database:** stores users, organisations, document sets, file metadata, extracted elements, links, edit operations, reviews, versions, and audit events;
8. **Private file storage:** stores original and generated files outside public access;
9. **External integration adapter:** imports or exports files through the selected cloud-storage service;
10. **Documentation website:** publishes technical and project documentation.

8.3 High-Level Data Flow

1. The authenticated user uploads documents through the frontend.
2. The frontend sends the files to the team's backend.
3. The backend validates file type, size, ownership, and safety.
4. Original files are stored privately and remain unchanged.
5. The parser extracts supported document elements.
6. Extracted metadata is stored in the relational database.
7. The matching engine identifies exact matches and similarity suggestions.
8. The user confirms which elements should be linked.

9. The user submits a proposed shared edit.
10. The backend validates permissions and creates a preview.
11. The user reviews all affected files and confirms the operation.
12. The edit engine creates new document versions.
13. Generated files, version metadata, and audit events are stored.
14. The user downloads the updated files or exports them through the external integration.

9 Preliminary Database Design

The relational database will store metadata and workflow state. Large original and generated files should be stored in private file or object storage, with secure references kept in the database.

Expected entities include:

- **User**: identity reference, display name, status, and account metadata;
- **Organisation**: tenant boundary and organisation settings;
- **Membership**: user role within an organisation;
- **DocumentSet**: a named collection of related uploaded files;
- **Document**: logical identity, original file name, media type, checksum, and status;
- **DocumentVersion**: immutable original or generated file version with storage reference;
- **DocumentElement**: extracted paragraph, heading, list item, table cell, or slide element;
- **LinkGroup**: user-confirmed set of elements that should be edited together;
- **MatchSuggestion**: exact or similarity-based proposed relationship with confidence metadata;
- **EditOperation**: proposed replacement, affected link group, status, and actor;
- **PreviewResult**: summary of expected changes and validation warnings;
- **Review**: approval or rejection decision and optional comment;
- **GenerationJob**: processing status, failures, timestamps, and resulting versions;
- **AuditEvent**: actor, action, affected resource, timestamp, and safe metadata.

The frontend will never receive database credentials or direct file-storage credentials. All access will pass through the backend so that validation, authorisation, tenant isolation, audit recording, and business rules are consistently enforced.

10 Security and Privacy

Uploaded documents may contain confidential, personal, legal, commercial, educational, or operational information. Security is part of the core scope.

- Authentication will use an established provider or maintained library.
- Authorisation will be enforced by the backend for every document and operation.
- Organisation data will be isolated using tenant identifiers and access checks.
- Files will be private by default and accessed using short-lived or backend-mediated access.

- File types, extensions, media types, sizes, and archive contents will be validated.
- Password-protected, malformed, or unsupported files will be rejected safely.
- File names and storage paths will be sanitised.
- Original uploads will be immutable.
- Secrets will be stored in deployment secret managers or environment variables.
- Audit logs will avoid unnecessary sensitive document content.
- Deployed environments will use encrypted transport.
- Retention, deletion, backup, and recovery procedures will be documented.
- External integrations will use least-privilege permissions.
- The system will not claim to verify the legal, medical, financial, or regulatory correctness of edited content.

11 Project Methodology

The team will use an adapted Agile/Scrum process aligned with the assessed milestones.

- Maintain a prioritised product backlog containing user stories, risks, defects, technical work, testing, documentation, and research spikes;
- Plan work in milestone-aligned sprints;
- Assign every task an owner, priority, dependencies, acceptance criteria, evidence, and target milestone;
- Hold recurring check-ins to identify progress and blockers;
- End each sprint with a review, stakeholder demonstration, and retrospective;
- Record important technical decisions using architecture decision records;
- Require pull-request review and successful CI checks before merge;
- Track defects by severity;
- Maintain weekly contribution and AI-use records.

11.1 Definition of Done

A task is complete only when:

- its acceptance criteria are met;
- the feature works on representative files;
- relevant tests pass;
- code has been reviewed;
- CI passes;
- documentation is updated;
- security, accessibility, responsiveness, validation, and error handling have been considered;
- the feature is deployed or demonstrably deployment-ready.

12 Milestone-Aligned Development Plan

12.1 Milestone 1: Foundations

- Create repositories, branch protections, issue templates, pull-request templates, and initial CI;
- Publish READMEs, setup instructions, and the public documentation website;
- Document Git and project methodologies;
- Gather representative sample documents and record confirmed stakeholder requirements;
- Create wireframes and an accessible interface foundation;
- Produce architecture, database, storage, deployment, and security designs;
- Prototype DOCX text extraction and reconstruction;
- Measure formatting preservation on real sample documents;
- Prototype exact paragraph matching;
- Implement a vertical slice: authentication, upload, parse, display, confirm one match, edit, preview, and download an updated DOCX;
- Record risks, decisions, prototype results, and the roadmap.

12.2 Milestone 2: Core Product

- Implement document sets, secure uploads, extraction, exact matching, confirmed link groups, editing, preview, and versioned downloads;
- Deploy the backend and document its hand-written HTTP interface;
- Implement the external cloud-storage integration;
- Add automated tests for document processing, backend workflows, and core UI journeys;
- Conduct formal user testing and a stakeholder review;
- Evaluate and integrate prioritised feedback;
- Use the bug tracker continuously;
- Complete database, storage, third-party, testing, and deployment documentation.

12.3 Milestone 3: Stabilisation

- Complete reliable DOCX support and selected Intermediate features;
- Add similarity suggestions only if exact matching is stable;
- Stabilise formatting preservation and failure handling;
- Fix all severe defects and reduce remaining defects;
- Expand regression tests using representative sample files;
- Measure and improve processing performance;
- Conduct accessibility and responsive-design reviews;
- Run further user testing and record measurable improvements;
- Complete production, security, recovery, and operational documentation.

12.4 Final Submission

- Deploy the frontend, backend, database, private storage, and documentation website;
- Verify registration, login, password reset, account deletion, authorisation, and failure paths;
- Verify supported upload, preview, edit, version, and download workflows;
- Verify original files remain unchanged;
- Verify backend documentation and external integration behaviour;
- Run complete automated tests and production smoke tests;
- Review secrets, storage permissions, accessibility, responsiveness, performance, backups, and final documentation accuracy;
- Preserve evidence for the report, presentation, and peer review.

13 Initial User Stories

13.1 Upload Existing Documents

As a document editor, I want to upload my existing related files, so that I can update them without recreating them.

Acceptance criteria:

- The user can upload multiple supported DOCX files.
- Unsupported, corrupt, or oversized files produce clear errors.
- Original files are stored unchanged.
- The user can view the uploaded files in a named document set.
- Access is restricted to authorised users.

13.2 Confirm Shared Content

As a document editor, I want to confirm which paragraphs are related, so that the system does not edit unrelated content.

Acceptance criteria:

- Exact matching content is highlighted.
- The user can select or reject proposed matches.
- No content is linked without an explicit user action.
- Confirmed links are stored for later use.
- The action is recorded in the audit history.

13.3 Edit Once Across Files

As a document editor, I want to edit confirmed shared content once, so that every selected document receives the same update.

Acceptance criteria:

- The editor can enter replacement content for a confirmed link group.

- The system lists every affected file and location.
- Unlinked content remains unchanged.
- Document-specific details remain intact.
- The user must preview the result before final generation.

13.4 Download Updated Versions

As a document editor, I want to download updated copies, so that I can continue using the edited documents while retaining the originals.

Acceptance criteria:

- New versions are generated only after confirmation.
- Original files remain unchanged and available according to retention policy.
- Updated files retain supported formatting.
- Generation failures are reported clearly.
- Version and audit records are stored.

14 Testing Strategy

- **Unit tests:** matching, replacement selection, validation, permissions, checksums, and version selection;
- **Document fixture tests:** extraction and reconstruction using representative DOCX files;
- **Regression tests:** verify that unchanged content and supported formatting remain unchanged;
- **Database tests:** migrations, constraints, relationships, tenant isolation, and transaction behaviour;
- **Backend integration tests:** authentication, upload, extraction, matching, edit preview, generation, versioning, and errors;
- **UI tests:** uploads, match confirmation, editing, previews, keyboard use, loading states, and error messages;
- **End-to-end tests:** registration, upload, confirm match, edit, preview, generate, download, and account deletion;
- **Security tests:** unauthorised file access, invalid archives, unsafe file names, tenant isolation, secret scanning, and dependency review;
- **Accessibility tests:** automated checks plus manual keyboard and screen-reader-oriented review;
- **Performance tests:** processing time and memory use for representative files and document sets;
- **User testing:** realistic editing tasks, observed problems, severity ratings, prioritised feedback, and evidence of improvements.

15 Risks and Mitigations

Risk	Impact	Mitigation
DOCX formatting is not preserved correctly	Generated files may be unusable or visually inconsistent	Prototype with real sample files during Milestone 1. Support a controlled subset first and create regression fixtures.
PowerPoint support is more complex than expected	The team may lose time and fail to stabilise the core product	Treat PPTX as Intermediate or Advanced scope. Complete reliable DOCX support first.
Similar paragraphs are matched incorrectly	Unrelated content may be edited	Use exact matching first. Treat similarity as a suggestion only and require explicit user confirmation.
A bulk edit changes the wrong files	High trust and operational impact	Show impact previews, require confirmation, preserve originals, create immutable versions, and support rollback.
Uploaded documents contain malicious or malformed content	Security or availability risk	Validate file types and archive contents, impose limits, reject unsupported files, isolate processing, and avoid executing embedded content.
Confidential documents are exposed	Serious privacy and security impact	Use established authentication, backend authorisation, tenant isolation, private storage, least privilege, secure secrets, and documented retention.
Direct frontend access bypasses backend controls	Security and compulsory-requirement violation	Route all database and file operations through the team's backend. Never expose storage or database credentials to the browser.
The scope becomes too broad	The final product may be unstable	Prioritise one end-to-end DOCX workflow. Add tables, similarity, PPTX, and advanced approvals only after the core is stable.
Other coursework reduces development capacity	Risky work may remain unfinished late in the semester	Prototype parsing, reconstruction, formatting preservation, authentication, storage, and deployment during Milestone 1.

16 Alternatives Considered

16.1 Manual Search and Replace

Manual editing requires users to open every file and does not provide reliable cross-document preview, audit history, versioning, or protection against missed updates.

16.2 Rebuilding Documents as Templates

A template system is useful for future documents but requires existing files to be recreated or converted. The proposed product focuses first on the client's actual need: editing documents that already exist.

16.3 Fully Automatic Matching and Editing

Automatically deciding that similar paragraphs are equivalent is unsafe. The system will generate suggestions, but users will confirm links before any shared edit is applied.

16.4 Direct Database or Storage Access From the Frontend

This could simplify a prototype, but it weakens validation, authorisation, auditability, and tenant isolation. It may also conflict with the requirement for a hand-written backend. The selected design uses a database and private storage through the team's backend.

17 Expected Deliverables

- Deployed responsive frontend;
- Deployed hand-written backend;
- Deployed relational database;
- Secure private file storage;
- Public documentation website;
- Team-maintained backend interface documentation;
- Architecture, database, storage, and processing diagrams;
- Setup, testing, deployment, security, accessibility, and recovery documentation;
- Automated tests and CI/CD workflows;
- Stakeholder requirements and review evidence;
- User-testing plan, results, evaluation, and improvements;
- Git and project-methodology evidence;
- AI-use register;
- Final report and presentation evidence.

18 AI Use and Attribution

AI tools may be used for planning, technical guidance, documentation, editing, test suggestions, or code assistance. Every use incorporated into the project will be recorded with the tool, model, date, team member, purpose, brief task, output used, and how it was checked or changed.

For work originating from this proposal drafting session:

- **Tool:** ChatGPT Web
- **Model:** GPT-5.6 Thinking
- **Purpose:** project planning, proposal drafting, technical guidance, and editing
- **Verification:** reviewed and adapted by the project team

The team remains responsible for understanding, checking, adapting, testing, and securely reviewing all AI-assisted work.

19 Conclusion

DocumentSync is a feasible and suitably challenging software-engineering project when scoped around the safe editing of existing related documents. Its main value is that users can upload the files they already have, confirm which content is shared, make one controlled change, preview the impact, and download updated versions without losing document-specific information.

The project is general enough to support different topics, organisations, and use cases. At the same time, the initial scope remains practical by prioritising DOCX, exact matching, user confirmation, immutable originals, previews, and versioned output.

The proposal supports the compulsory course requirements and provides a credible path to the Advanced tier. The highest risks—format preservation, incorrect matching, unsafe bulk editing, confidential files, and excessive scope—will be addressed early through prototypes, representative test files, user confirmation, automated regression tests, secure architecture, and milestone-based prioritisation.

A Initial Acceptance Test Scenario

Scenario: Update one paragraph across three existing DOCX files

1. Given an authorised user has uploaded three related DOCX files;
2. and each file contains the same reporting paragraph;
3. and each file also contains different names and addresses;
4. when the system identifies the repeated paragraph;
5. and the user confirms that the three paragraphs should be linked;
6. and the user enters revised wording;
7. then the system shows a preview for all three files;
8. and the preview changes only the confirmed paragraph;
9. and all unique names, addresses, and unrelated content remain unchanged;
10. when the user confirms generation;
11. then the system creates three new downloadable DOCX versions;
12. and the original files remain unchanged;
13. and the edit, actor, timestamps, source versions, and generated versions are recorded.

B Items Requiring Stakeholder Confirmation

- Representative existing documents that may be used for development and testing;
- Whether the first release must support DOCX only or both DOCX and PPTX;
- Which formatting features must be preserved;
- Whether tables, images, headers, footers, numbering, comments, or tracked changes are required;
- Typical and maximum file sizes;
- Typical number of files in one document set;

- Whether exact matches are sufficient for the first version;
- Whether users need placeholders for unique values;
- Who may upload, edit, review, approve, download, and delete files;
- Required retention period for originals, generated versions, and audit history;
- Preferred external cloud-storage integration;
- Whether documents contain confidential, personal, or regulated information;
- Whether macros, password-protected files, electronic signatures, or collaborative editing are explicitly out of scope.